Hamming Code

Main Structure

P1	P2	D1	P4	D2	D3	D4
1	2	3	4	5	6	7

Red: Parity Bit

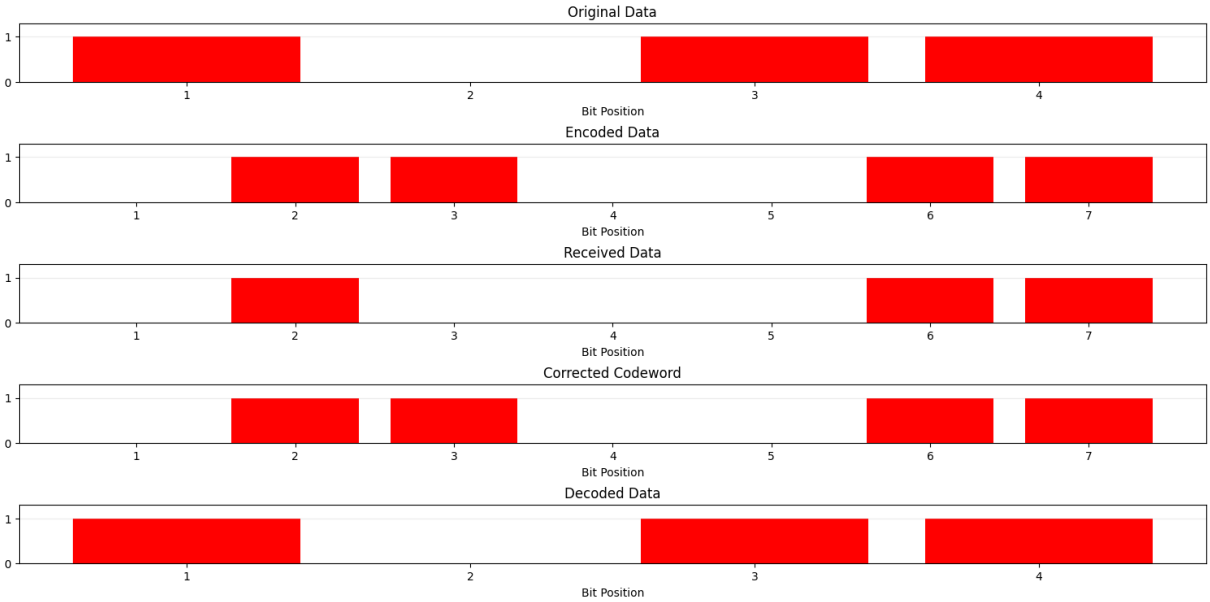
Blue: Data Bit

By initiating the input or original data as follows:

Stage	Binary result
Original data	1011
Encoded data	0110011
Received data (error at bit 3)	0100011
Syndrome	011 (position 3 in binary)
Corrected codeword	0110011
Decoded data	1011

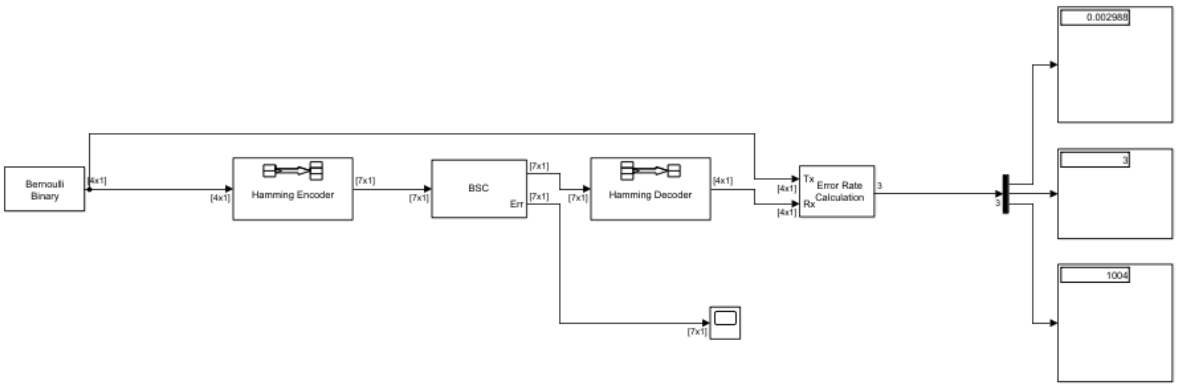
The following result will be yield

Output



Original Data: [1 0 1 1]
Encoded Data: [0 1 1 0 0 1 1]
Received Data: [0 1 0 0 0 1 1]
Syndrome: 011
Error Position: 3
Corrected Data: [0 1 1 0 0 1 1]
Decoded Data: [1 0 1 1]
Successful: True

Simple MATLAB Simulink Approach



Annex: Python Script

```
# HAMMING (7,4) ENCODER AND DECODER
import numpy as np
import matplotlib.pyplot as plt

def encode_hamming(data):
    """Even Parity"""
    if len(data) != 4:
        raise ValueError("Input must contain exactly 4 bits.")

    c = np.zeros(7, dtype=int)
    c[[2, 4, 5, 6]] = data

    # Calculate parity bits
    for p in [1, 2, 4]:
        indices = [i - 1 for i in range(1, 8) if i & p]
        c[p - 1] = np.sum(c[indices]) % 2

    return c


def decode_hamming(received):
    """Detection like a geek!"""
    corrected = received.copy()
    syndrome = 0

    for p in [1, 2, 4]:
        indices = [i - 1 for i in range(1, 8) if i & p]
        parity_check = np.sum(received[indices]) % 2

        if parity_check:
            syndrome += p

    if syndrome != 0:
        corrected[syndrome - 1] ^= 1

    decoded = corrected[[2, 4, 5, 6]]
    return corrected, decoded, syndrome

data = np.array([1, 0, 1, 1])
```

```
error_position = 3  # 1-based; set to 0 while no error exist baby!

encoded = encode_hamming(data)

received = encoded.copy()
if error_position != 0:
    received[error_position - 1] ^= 1

corrected, decoded, syndrome = decode_hamming(received)

print("Original Data:    ", data)
print("Encoded Data:     ", encoded)
print("Received Data:    ", received)
print("Syndrome:         ", format(syndrome, "03b"))
print("Error Position:   ", syndrome)
print("Corrected Data:   ", corrected)
print("Decoded Data:     ", decoded)
print("Successful:       ", np.array_equal(data, decoded))

stages = [
    ("Original Data", data),
    ("Encoded Data", encoded),
    ("Received Data", received),
    ("Corrected Codeword", corrected),
    ("Decoded Data", decoded)
]

fig, axes = plt.subplots(len(stages), 1, figsize=(10, 8))

for ax, (name, bits) in zip(axes, stages):
    colors = ["red" if b else "steelblue" for b in bits]
    ax.bar(np.arange(1, len(bits) + 1), bits, color=colors)
    ax.set_title(name)
    ax.set_ylim(0, 1.3)
    ax.set_yticks([0, 1])
    ax.set_xticks(np.arange(1, len(bits) + 1))
    ax.set_xlabel("Bit Position")
    ax.grid(axis="y", alpha=0.25)

plt.tight_layout()
plt.show()
```